

Motor de Búsqueda para Ofertas Laborales: Comparación de Modelos Clásicos y Semánticos

Mateo Sebastián Cumbal Guasgua Daniel Ismael Flores Espín
Alicia Dayana Pereira Tuqueres

2026-06-01

1. Descripción del Corpus

El corpus se construyó a partir de ofertas laborales recolectadas mediante scraping de las plataformas **Jooble** y **LinkedIn** (vía RapidAPI y Coresignal) para **24 carreras universitarias** de la Escuela Politécnica Nacional. El pipeline de obtención incluyó traducción automática al español, normalización de texto y extracción de habilidades blandas con el framework EURACE.

Table 1: Estadísticas generales del corpus

Característica	Valor
Total de documentos únicos	74,517
Vocabulario (términos únicos)	57,057
Campos indexados	<code>job_title</code> , <code>description_final</code> , <code>careers_required</code>
Idioma del corpus	Español (traducido)
Carreras cubiertas	24 carreras universitarias

Cada documento tiene un identificador único (`job_id` tipo SHA-256) y está asociado a una o más carreras a través del campo `all_careers`, que sirve como *ground truth* para la evaluación.

2. Decisiones de Diseño

2.1 Preprocesamiento

Se aplicó un pipeline uniforme a todas las consultas y documentos: (1) conversión a minúsculas y eliminación de tildes mediante normalización Unicode NFD, (2) remoción de stopwords en español con NLTK, y (3) stemming con **SnowballStemmer** para español. La función `limpiar_texto()` en `preprocessing.py` actúa primero, seguida de `ProcesadorNLP.tokenizar()` en `indexer.py`. Este enfoque compartido garantiza que las consultas y los documentos sean comparables en el mismo espacio de representación.

2.2 Índice Invertido

Se implementó desde cero en `indexer.py` con estructura `dict[token → dict[doc_id → freq]]`. Se almacenan adicionalmente la longitud de cada documento (`longitud_docs`) y el total de documentos

(`total_documentos`), datos requeridos por TF-IDF y BM25. El índice se serializa con `pickle` para evitar reconstruirlo en cada sesión.

2.3 Elección de Modelos

Se implementaron tres modelos clásicos y uno semántico:

- **Jaccard** — similitud de conjuntos binarios. Sin pesos, útil como *baseline*.
- **Coseno TF-IDF** — captura frecuencia relativa y rareza del término. Las normas de documentos se precomputan al inicializar para eficiencia.
- **BM25** — variante probabilística que satura el TF y normaliza por longitud de documento. Parámetros: $k_1 = 1.2$, $b = 0.75$.
- **Embeddings densos** — modelo `paraphrase-multilingual-MiniLM-L12-v2` (sentence-transformers) con ChromaDB como base de datos vectorial. El score se define como $1/(1 + d_{L2})$.

2.4 Evaluación

Se construyeron *qrels* automáticos: cada documento se asocia a las carreras que aparecen en su campo `all_careers`. Las 24 consultas de prueba se diseñaron manualmente para reflejar búsquedas reales por perfil profesional (ej. “*cientifico de datos data science big data analisis estadistico machine learning*”).

3. Ejemplos de Consultas y Resultados

La siguiente tabla muestra los top-3 resultados para la consulta “**analista de datos python**” con cada modelo.

Table 2: Top-3 resultados para la consulta «analista de datos python»

#	Jaccard	Coseno TF-IDF	BM25	Embeddings
1	staff data analyst (0.133)	data analyst python programming work from home (0.562)	analista pleno netbackup e tsm (15.93)	data analyst python (...)
2	data scientist fort meade (0.111)	data analyst (0.436)	data analyst python programming work from home (15.92)	analista de datos senior
3	data analyst python programming work from home (0.107)	job opportunity data analyst (0.423)	analista pleno de ia generativa (14.64)	analista de datos

Observaciones:

- **Jaccard** retorna resultados como “*staff data analyst*” en primer lugar (score 0.133) por coincidencia de conjunto, sin considerar la rareza del término **python** en el corpus, lo que lo hace impreciso.
- **TF-IDF** posiciona correctamente el documento que contiene los tres términos de la consulta en primer lugar (0.562) gracias al peso IDF elevado de **python** (presente en solo 2,147 documentos de 74,517).

- **BM25** también identifica el documento más relevante, pero prioriza documentos de menor longitud donde los términos tienen mayor densidad relativa, lo que explica el primer lugar de “*analista pleno netbackup*” — que contiene **analista** con alta frecuencia en un texto corto.
- **Embeddings** captura semánticamente perfiles relacionados aunque no coincidan token a token.

4. Análisis de Métricas de Evaluación

Se evaluaron los cuatro modelos sobre las 24 consultas usando **Precision@10**, **Recall@10** y **MAP** (Mean Average Precision). Los *qrels* se generaron a partir de la columna `all_careers` del corpus.

Table 3: Métricas de evaluación sobre 24 consultas — *qrels* ampliados (etiqueta + título)

Modelo	Precision@10	Recall@10	MAP
Jaccard	0.2042	0.0042	0.0054
TF-IDF Coseno	0.3042	0.0120	0.0158
BM25	0.2292	0.0037	0.0071
Embeddings Densos	0.3750	0.0283	0.0419

Análisis:

Los *qrels* combinan dos fuentes de relevancia: la etiqueta exacta de carrera en `all_careers` y la coincidencia de términos asociados en `job_title`, lo que reduce el sesgo contra modelos que recuperan sinónimos léxicos sin compartir tokens exactos con la etiqueta.

Jaccard obtiene el peor MAP (0.0054) pese a una Precision@10 razonable (0.2042). La discrepancia refleja su incapacidad para ordenar: recupera algunos relevantes entre los 10 primeros, pero sin una señal de ranking confiable — el orden es casi arbitrario, lo que castiga severamente el MAP.

TF-IDF mejora en todos los indicadores (MAP 0.0158, +193% sobre Jaccard). El peso IDF discrimina términos raros y mejora tanto la precisión como el orden de los resultados. Es el segundo mejor modelo clásico en MAP y Precision@10.

BM25 sorprende al quedar por debajo de TF-IDF en este experimento (MAP 0.0071). Con *qrels* ampliados, el corpus de documentos relevantes es más heterogéneo en longitud; la normalización por longitud de BM25 ($b = 0.75$) penaliza documentos largos que, en este contexto, son precisamente los que contienen mayor variedad de términos sinónimos y por tanto más probabilidad de ser relevantes.

Embeddings densos lidera en las tres métricas (MAP 0.0419, Precision@10 0.3750, Recall@10 0.0283). La ventaja es especialmente notable en Recall@10: recupera $7.6\times$ más documentos relevantes que BM25 en los primeros 10 resultados. Esto confirma la hipótesis central: al proyectar consultas y documentos en un espacio semántico de 384 dimensiones, el modelo captura relaciones conceptuales — “*data scientist*”, “*analista de datos*” e “*ingeniero de datos*” convergen en el mismo vecindario vectorial — que son invisibles para cualquier modelo basado en coincidencia léxica.